

Flow:

1) find dataset
2) embedding
3) convert to

1. Dataset Section

Loads question-answer data.

2. Text Preprocessing (Tokenization)

```
def tokenize(text):
```

```
    text = text.lower()

    text = text.replace('?', '')

    text = text.replace("'", "")

    return text.split()
```

✔ What it does:

- Cleans text
- Converts sentence → list of words

Example:

"What is France?" → ["what", "is", "france"]

💡 Why necessary:

- Neural networks **cannot understand raw text**
- **Must convert → structured format (tokens)**

3. Vocabulary Building

```
vocab = {'<UNK>': 0}
def build_vocab(row):
    ...
```

✔ What it does:

- Assigns each word a unique number

✔ Example:

"what" → 1

"is" → 2

"france" → 53

💡 Why necessary:

- Models only understand **numbers, not words**
- Vocabulary = **bridge between text & numbers**

4. Text → Numerical Conversion

```
def text_to_indices(text, vocab):
```

✔ What it does:

Converts words → numbers

Example:

"What is France" → [1, 2, 53]

💡 Why necessary:

- Required input format for PyTorch
- Handles unknown words using <UNK>

5. Custom Dataset Class

```
class QADataset(Dataset):
```

✔ What it does:

- Wraps dataset into PyTorch format
- Returns:
 - question tensor
 - answer tensor

💡 Why necessary:

- PyTorch training requires **Dataset objects**
- Enables batching, shuffling, iteration

6. DataLoader

```
dataloader = DataLoader(dataset, batch_size=1, shuffle=True)
```

✔ What it does:

- Feeds data to model in batches

•

💡 Why necessary:

- Efficient training
- Randomization improves learning

7. Model Architecture (Simple RNN)

```
class SimpleRNN(nn.Module):
```

• Components:

(a) Embedding Layer

```
self.embedding = nn.Embedding(vocab_size, 50)
```

✔ Converts word index → dense vector

💡 Why:

- Words → meaningful vector representation

(b) RNN Layer

```
self.rnn = nn.RNN(50, 64, batch_first=True)
```

✔ Processes sequence step-by-step

- Why:
- Captures **sequence information**
- Understands **word order**

```
(c) Fully Connected Layer

self.fc = nn.Linear(64, vocab_size)
```

- Converts hidden state → output prediction
- Why:
- Maps RNN output → vocabulary space
- Predicts answer word

```
8. Forward Pass
def forward(self, question):
```

Flow:

Question → Embedding → RNN → Final Hidden → FC → Output

- Why necessary:
- Defines how data flows through model

9. Training Setup

```
criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.Adam(...)
```

- What it does:
- Loss: measures error
- Optimizer: updates weights
- Why necessary:
- Without loss → no learning signal
- Without optimizer → no improvement

10. Training Loop

```
for epoch in range(epochs):
```

- Flow:
- Forward pass
 - Compute loss
 - Backpropagation
 - Update weights
 -

11. Prediction Function

```
def predict(model, question):
```

- What it does:
- Converts input question
- Runs model
- Outputs predicted answer
- Why necessary:
- Allows real-world usage after training

12. Softmax + Argmax

```
probs = softmax(output)
value, index = torch.max(...)
```

- What it does:
- Converts scores → probabilities
- Picks best answer
-
- Why necessary:
- Model outputs raw scores (logits)
- Need probabilities to interpret

FINAL PIPELINE (IMPORTANT)

Dataset

↓

0. Overall Concept

👉 This function is doing **Text Preprocessing + Tokenization**

Vocabulary

↓

1. Embedding

👉 "Convert raw messy text → clean list of words (tokens)"

Dataset: Class + DataLoader

Example:

Embedding: "What is a?" → ["what", "is", "a"]

↓

RNN Layer

↓

1. Lowercasing

↓

Fully Connected Layer

↓

text = text.lower()

↓

Loss + Optimizer

↓

Training Loop

↓

Why needed:

Prediction

- "France" and "france" should be treated as **same word**
-
- Otherwise vocabulary becomes:
- France + france ❌

- Benefit:
- Reduces vocabulary size
- Improves model learning

2. Remove Quotes

```
text = re.sub(r'["']', "", text)
```

- Theory:
- This is **text cleaning (noise removal)** using pattern matching

- r"[""]" → matches:
- " (double quote)
- ' (single quote)
-

- Why needed:
- Quotes don't carry meaning in QA tasks

Example:

"To Kill a Mockingbird" → To Kill a Mockingbird

- ✔ Benefit:
- Prevents tokens like "mockingbird" vs mockingbird

3. Remove Punctuation

```
text = re.sub(r"[^a-z0-9\s]", "", text)
```

Theory:

This is **filtering unwanted characters** using regex

- $\wedge \rightarrow$ NOT
- a-z $\rightarrow$ letters
- 0-9 $\rightarrow$ numbers
- \s $\rightarrow$ spaces
-  So this removes **everything except letters, numbers, spaces**

Why needed:

Example:

"What!!! is this?" $\rightarrow$ "what is this"

Without this:

"what!!!" becomes a token ❌

✔ Benefit:

- Clean tokens
- Better generalization

4. Remove Extra Spaces

```
text = re.sub(r"\s+", " ", text).strip()
```

Theory:

This is **whitespace normalization**

- \s+ $\rightarrow$ multiple spaces $\rightarrow$ replaced with single space
- .strip() $\rightarrow$ removes spaces from start & end

Why needed:

After cleaning:

"what is this" $\rightarrow$ "what is this"

✔ Benefit:

- Prevents empty tokens
- Cleaner splitting

5. Tokenization

```
tokens = text.split()
```

Theory:

This is **word-level tokenization**

-  Converts sentence $\rightarrow$ list of words

Why needed:

Neural networks need structured input:

"what is ai" $\rightarrow$ ["what", "is", "ai"]

✔ Benefit:

- Each word becomes a unit for:
- Vocabulary
- Embedding
- RNN processing

FINAL PIPELINE THEORY

```
Raw Text
↓
Lowercasing
↓
Remove Quotes
↓
Remove Punctuation
↓
Normalize Spaces
↓
Tokenization
↓
List of Clean Words
```

vocab

vocab = { 'UNK': 0 } = ① 2 3 4 5
sentence, answer

unique

'what' is 1
is 2
the 3
capital 4

What is the capital of France paris
run on = len(vocab) the capital is 5

vocab (tokens) —

text to index

what is phitron

[1 2 0]

~~france~~
paris

france
paris [7]

germany [8]

~~Italy~~ → berlin

Architecture

encoder
sent padding

✓ High-Level Idea

This model is a **sequence** → **single output model**.

- Input: a **question (sequence of words)**
- Output: a **single word (answer)**

So the pipeline is:

Text → Numbers → Embedding → RNN → Final Hidden State → Linear → Prediction

Step	Representation	Explanation
Input Text	"What is France?"	Original sentence
Tokenized Input	[12, 5, 87]	Each word converted to index
After Embedding	[[0.2, -0.1, ...], [0.5, 0.3, ...], [0.1, -0.7, ...]]	Each index → 50-dimensional vector

◆ 2. Model Components

✓ (1) Embedding Layer

self.embedding = nn.Embedding(vocab_size, embedding_dim)

What it does:

- Converts **word indices** → **dense vectors**
-
- Each word becomes a vector of size embedding_dim = 50
-

Example:

"What is France?" → [12, 5, 87]

↓

Embedding

↓

[[0.2, -0.1, ...], ← 50-dim vector
[0.5, 0.3, ...],
[0.1, -0.7, ...]]

Shape:

Input: (1, seq_len)
Output: (1, seq_len, 50)

1. Model Overview

Component	Purpose	Input Shape	Output Shape
Embedding	Convert word indices → dense vectors	(1, seq_len)	(1, seq_len, 50)
RNN	Process sequence & capture context	(1, seq_len, 50)	(1, seq_len, 64)
Final Hidden State	Summary of full sentence	(1, seq_len, 64)	(1, 1, 64)
Fully Connected (Linear)	Map hidden → vocabulary scores	(1, 64)	(1, vocab_size)

*** ⚡ batch_first=True

(self.rnn = nn.RNN(embedding_dim, hidden_size, batch_first=True))

means the input shape starts with batch size.

So:

(batch_size, sequence_length, features)

Instead of:

(sequence_length, batch_size, features)

⚡ In short: batch comes first.

*** ⚡ Sequence length = number of time steps (or words) in an input sentence.

💡 Simple meaning

It tells:

"How many tokens (words) are in the sequence?"

📌 Example

Sentence:

"I love deep learning"

After tokenization:

["I", "love", "deep", "learning"]

⚡ Sequence length = 4

📦 In RNN input

If:

- batch_size = 1
- sequence_length = 4
- embedding_dim = 50

Shape becomes:

(1, 4, 50)

◆ 2. Forward Pass Step-by-Step

Step	Code	What Happens	Shape
1	embedded = self.embedding(question)	Convert words → vectors	(1, seq_len, 50)
2	→, final = self.rnn(embedded)	Process sequence, keep last hidden state	(1, 1, 64)
3	final.squeeze(0)	Remove extra dimension	(1, 64)
4	self.fc(...)	Convert to vocabulary scores	(1, vocab_size)

🔍 In our code:


```
_, final = self.rnn(embedded)
```

Why two outputs?

nn.RNN() returns:
(output, hidden_state)

- So:
- output → hidden states at all time steps
 - hidden_state → final hidden state (last step)

Why _?

Because you **don't need the first output**, so you ignore it.
_ = output # we don't use it

What you actually use

You use:
final

This contains the **last hidden state**, which summarizes the whole sequence.

3. Data Flow Pipeline

Stage	Description	Shape
Input	Question as word indices	(1, seq_len)
Embedding	Dense vector representation	(1, seq_len, 50)
RNN Processing	Sequence understanding	(1, seq_len, 64)
Final State	Sentence summary	(1, 64)
Output Layer	Word prediction scores	(1, vocab_size)

4. Shape Transformation Summary

	Layer	Input Shape	Output Shape
	Input	(1, seq_len)	—
	Embedding	(1, seq_len)	(1, seq_len, 50)
✓	RNN	(1, seq_len, 50)	(1, 1, 64)
	Squeeze	(1, 1, 64)	(1, 64)
	Linear	(1, 64)	(1, vocab_size)

5. Key Concepts

Concept	Explanation
Embedding	Converts discrete words into continuous vectors
RNN	Processes sequence step-by-step and keeps memory
Final Hidden State	Represents the whole sentence
Linear Layer	Maps hidden features to vocabulary scores
Output	Logits (unnormalized probabilities)

6. Example Flow

Step	Example
Input	What is the capital of France?"
After Embedding	Sequence of vectors
After RNN	Context captured step-by-step
Final Hidden State	Encodes full meaning
Output	"Paris"